

Report

0.1

Customer

Epineon



NAPQUES v2 — AEAD Scheme Audit

July 19, 2026

Contents

1. Document Revisions	3
2. Introduction	3
3. System Overview	4
4. Methodology	5
5. Findings	6
5.1. Major.....	7
5.2. Moderate.....	11
5.3. Minor.....	13
6. Conclusion	22
About Us	24
Contacts	24

1. Document Revisions

#	Date	Author	Description
0.1	July 19, 2026	Dmitry Khovratovich	Initial v2-round report. Reconciled all 32 first-round

2. Introduction

ALL MODIFICATIONS TO THIS DOCUMENT ARE PROHIBITED. VIOLATORS WILL BE PROSECUTED TO THE FULL EXTENT OF THE U.S. LAW.

Client Comment

IT IS IMPORTANT TO NOTE THAT A SECURITY AUDIT IS NOT A GUARANTEE OF ABSOLUTE SECURITY BUT RATHER A SNAPSHOT OF THE SYSTEM'S SECURITY POSTURE AT A SPECIFIC POINT IN TIME. WHILE WE STRIVE TO UNCOVER ALL POTENTIAL ISSUES, THE EVOLVING NATURE OF BLOCKCHAIN TECHNOLOGY AND DEFI MEANS NEW VULNERABILITIES CAN EMERGE.

3. System Overview

NAPQES (Noise-Augmented Post-Quantum Encryption System) is an Authenticated Encryption with Associated Data (AEAD) scheme whose security is claimed to reduce entirely to the pseudorandomness of HMAC-SHA256, deliberately avoiding the finite-field and permutation structures used by AES-GCM and ChaCha20-Poly1305. A NAPQES key is an ordered tuple $\mathbf{k} = (k_1, \dots, k_K)$ of K distinct primes sampled uniformly without replacement from the set $\mathcal{P}$ of primes in $[10^6, 9.9 \times 10^6]$, serialised as the concatenation of five-byte big-endian encodings. All internal values are produced by a domain-separated derivation $\text{Derive}_d(\text{kb}, N, \text{ctx}) = \text{HMAC}(\text{kb}, d \| N \| \text{ctx})$, whose one-byte domain separator d selects among the noise-position, real-token addend, noise probability, authentication-tag, noise-character, noise-token addend, padding-codepoint and keystream functions (domains $0x00\text{--}0x07$), with further domains reserved for the streaming format and the version-8 synthetic nonce.

This revision restructures the scheme around two wire formats. The version-7 format encodes each padded codepoint as a fixed-width be8 prime-indexed token $c \cdot k + a$, interleaves pseudorandom noise tokens under a per-position probability $p \in [0.75, 0.99]$, and frames the masked token blob as $C = N \| \widetilde{B} \| T$ with a 16-byte nonce N and tag $T = \text{Derive}_{0x03}(\text{kb}, N \| \widetilde{B})$; the fixed-width token encoding is intended to remove the ciphertext-length channel present in the previous version. The recommended default is the version-8 misuse-resistant mode, which replaces the caller-supplied nonce with a synthetic nonce $N = \text{HMAC}(\text{sk}, 0x0A \| \text{be4}(|A|) \| A \| M)[0:16]$ derived deterministically from the associated data and message, so that accidental nonce reuse across distinct messages cannot occur.

The revision also expands the security analysis, adding a formal AEAD algorithm triple, an IND-CPA game, INT-CTXT and IND-CCA theorems, and a series of Audit finding remarks that fold in the first round's findings and record the corresponding fixes.

/ (root directory)

napseq-eprint-v2.tex

Client Comment

4. Methodology

The methodology utilized by ABDK Consulting is a combination of established audit tactics and methods specifically tuned for the requirements and technological stack of the NAPQES v2 — AEAD Scheme Audit project. This approach is flexible and adapted based on the specific project structure, technologies used, and the client's expectations for the security review.

Our audit process incorporates several distinct phases of analysis:

General Code Assessment

The codebase is first reviewed for clarity, consistency, style, and adherence to established best practices. This phase includes checking indentation standards, naming conventions (e.g., following a style guide), appropriate commenting, code duplication, and confusing or irrelevant naming. At this phase, we establish an understanding of the overall system structure and codebase organization.

Entity Usage Analysis

Usages of various entities (contracts, libraries, functions, state variables) defined in the code are analyzed. This includes both internal usages from other parts of the system as well as potential external interactions. We verify that entities are defined with appropriate visibility scopes (internal, external, public, private) and access levels. At this phase, we understand the overall system architecture and how different parts of the code are related and interact with external interfaces.

Access Control Analysis

For those entities that can be accessed externally or manage sensitive operations (like data submission or role management), access control measures are rigorously analyzed. We check that access control mechanisms are correctly implemented, relevant to the intended function, and robust against circumvention. At this phase, we understand the defined user roles, permissions logic, and what digital assets the system ought to protect.

Code Logic Analysis

The specific code logic of critical functions is analyzed for correctness, efficiency, and security. We verify that the code actually performs its intended function as described in the system overview, that algorithms are optimal for on-chain execution, and that proper data types are used to prevent overflow/underflow issues. We also ensure that external libraries used are up to date and relevant. At this phase, we understand the data structures used and their specific purposes.

We classify all identified issues uncovered during these phases by the following severity levels:

Critical

Directly affects the smart contract functionality and may cause a significant loss of assets (e.g., permanent lock of funds, unauthorized minting, total system compromise) or a complete failure of the platform's core invariants.

Major

Is either a solid performance problem or a sign of significant misuse potential: a slight code modification or environment change may lead to loss of funds or data integrity issues. Sometimes it is an abuse of unclear code behavior which should be double-checked by the client for specific edge-case business logic.

Moderate

Is not an immediate functional problem but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain expected business flows (lacking robust handling of all possible inputs).

Minor

Covers code style improvements, adherence to best practices that do not immediately present a security risk, and general suggestions for code clarity, gas efficiency, or future-proofing the system.

5. Findings

We have completed our technical audit of the NAPQES v2 — AEAD Scheme Audit codebase and provided the client with a detailed list of recommendations and identified vulnerabilities. The following statistics summarize the issues discovered and their current resolution status at the time of this report's finalization.

Major

Found

2

Fixed

0**Moderate**

Found

2

Fixed

0

MinorFound
10Fixed
0

Fixed 0 out of 14 issues reported.

5.1. Major

CVF-1: openCategory: **flaw**

The v8 IND-CPA theorem is false as stated, because v8 is a deterministic scheme while the notion it is proved against lets the adversary query the challenge message

The v8 encryption algorithm computes its nonce deterministically from the message. Concretely, the synthetic nonce is $N = \text{HMAC}(\text{sk}, 0x0A \parallel \text{be4}(|A|) \parallel A \parallel M)[0:16]$, and the rest of $\text{Enc}_{v8}(\mathbf{k}, \text{sk}, A, M)$ draws no additional randomness. As a consequence, the public encryption operation $\text{Encrypt}_{v8}(\mathbf{k}, \text{sk}, A, M)$ is a pure function of the pair (A, M) , which means that encrypting the same associated data and the same message a second time always returns exactly the same ciphertext.

The theorem "IND-CPA, v8" is nevertheless proved against the "IND-CPA security" definition without any modification to that definition. The definition is the standard left-or-right game, in which the adversary is given an encryption oracle both before and after it submits the challenge pair, the only constraint on the pair is that $|m_0| = |m_1|$ (equal codepoint count), and the text says in so many words that "no other restriction is imposed". In particular, nothing in the game prevents the adversary from asking the oracle to encrypt one of the two challenge messages.

Because the scheme is deterministic, this permission is enough to win the game outright. The adversary first asks the oracle for $y = \text{Encrypt}_{v8}(A^*, m_0)$ and remembers the answer. It then submits the challenge pair (A^*, m_0, m_1) with $|m_0| = |m_1|$, receives the challenge ciphertext c^* , and finally outputs the guess $b' = 0$ exactly when $c^* = y$. If the challenger encrypted m_0 , then c^* equals y with certainty; if it encrypted m_1 , then c^* differs from y except with negligible probability. The adversary therefore wins with advantage close to $1/2$, which is far larger than the value $\text{Adv}^{\text{PRF}} + q^2/2^{128}$ that the theorem claims to bound it by.

The exact place where the proof breaks down can be pointed to. The theorem describes Encrypt_{v8} as "already randomized, via the synthetic nonce", but a synthetic nonce is a

deterministic function of the input and so contributes no randomness whatsoever. The proof then reuses the hiding lemma, whose argument requires the challenge nonce to be fresh, meaning $N^* \notin \{N_1, \dots, N_q\}$, and it tries to guarantee that freshness from the collision bound. That step is unsound: the lemma "V8 closes CVF3" only bounds the probability that two distinct (A, M) pairs collide, and it says nothing about the case in which the challenge message was itself an earlier oracle query. In that case the challenge nonce coincides with a previously observed nonce with probability one rather than $q^2/2^{128}$, so the freshness hypothesis fails. Because the IND-CCA corollary is obtained by composition on top of this same IND-CPA claim, it inherits the identical defect.

Recommendation

Prove v8 against the security notion that actually matches a deterministic, misuse-resistant scheme instead of against the standard IND-CPA game. The natural choice is the deterministic / misuse-resistant authenticated-encryption notion of Rogaway and Shrimpton, which is exactly the notion under which AES-GCM-SIV is analysed. In that game the adversary is forbidden from repeating an (A, M) encryption query and from querying either challenge message, and that restriction is precisely what makes the challenge nonce fresh, so the existing proof skeleton goes through and $q^2/2^{128}$ becomes the correct collision term. If the authors instead wish to keep the standard IND-CPA statement, they must make Enc_{v8} genuinely randomized, for example by sampling a fresh random salt, folding it into the $0x0A$ nonce-derivation input, and transmitting it alongside the ciphertext. In either case this is a correction to the security definition and its proof, not a change to the construction itself.

Client Comment

/napseq-eprint-v2.tex

```

959 \begin{definition}[Synthetic nonce --- domain
    \texttt{0x0A}]\label{def:synthnonce}
972 \begin{definition}[V8 encryption and
    decryption]\label{def:aead-triple-v8}
1069 \begin{theorem}[IND-CPA, v8]\label{thm:ind-cpa-v8}
1075 via the synthetic nonce) public encryption oracle, and let
1114 Lemma~\ref{lem:v8-cvf3}, two queries collide in synthetic nonce only
    with
1126 \begin{corollary}[INT-CTXT and IND-CCA, v8]\label{cor:v8-security}
1320 \begin{definition}[IND-CPA security]\label{def:ind-cpa}

```

CVF-2: openCategory: **flaw****The v8 ciphertext length depends on plaintext content and, under chosen associated data, reliably reveals the plaintext length bucket**

In v8 the nonce is derived from the message, as $N = \text{HMAC}(\text{sk}, 0x0A \parallel \text{be4}(|A|) \parallel A \parallel M)[0:16]$. This means that two different messages of the same codepoint length are encrypted under two different nonces, and the noise layer is driven entirely by the nonce: both the noise probability p and every individual noise-or-real decision are pseudorandom values keyed by that nonce. The number of tokens N_{tok} therefore differs between the two messages, and since the ciphertext length is $|C| = 16 + 8N_{\text{tok}} + 32$, the ciphertext length itself becomes a fingerprint of the specific plaintext. A passive network observer who sees nothing but ciphertext sizes can already exploit this.

The paper's own traffic-analysis lemma cannot be used to rule this out. The lemma "Ciphertext length is decorrelated from plaintext content" is stated for two messages that are "encrypted under the same key and the same nonce N ". In v8 that hypothesis can never be satisfied for two distinct messages, because the nonce is a function of the message, so the lemma says nothing about the very situation in which the v8 proof invokes it. The proof nonetheless asserts that the lemma "applies verbatim ... only on it being fresh", and that assertion is incorrect.

The weakness becomes a concrete measurement as soon as the adversary can influence the associated data A , which is a mild assumption because associated data is usually headers or routing metadata rather than a secret. Suppose the adversary wants to

learn the length of a fixed target message M . Since A feeds only the nonce and the authentication tag and never the padding, the number of real tokens $R = B + 2$ is the same no matter which A is chosen, while each distinct A produces an independent nonce and therefore an independent quantity of noise. Encrypting M under many different values of A thus yields many independent samples $N_{\text{tok}} = R + W$ that all share the same R but have independent noise counts W . Averaging these samples cancels the noise, because the expected value is $\mathbb{E}[N_{\text{tok}}] \approx 13.4 R$ and the padding buckets give well-separated values $R \in \{18, 34, 66, 130, \dots\}$ that roughly double from one bucket to the next; only a modest number of samples is then needed to identify the bucket $B = \max(16, 2^{\lceil \log_2(n+1) \rceil})$ with confidence approaching certainty. In effect, the message-derived nonce turns the length-bucket leak that the paper already acknowledges as CAV-003 from a noisy, single-shot side channel into a reliable oracle, and it does so even for an application that only ever emits one ciphertext per (A, M) pair.

It is worth stating carefully what this attack does and does not recover. It recovers the padding bucket, which is essentially the order of magnitude of the length, namely $\lceil \log_2(n+1) \rceil$. It does not recover the exact codepoint count n , because the power-of-two padding maps every length in the range $[2^{j-1}, 2^j - 1]$ to the same number of real tokens and the same noise distribution, so the precise length is genuinely absent from the ciphertext size. Finally, this weakness is specific to v8: under v7 the nonce is random and independent of the message, so the challenger uses one and the same nonce for both challenge messages and their ciphertext lengths coincide. The failure here is a direct consequence of making the nonce a function of the message, and it is separate from the deterministic-equality break, because it harms even a purely passive observer who never interacts with the encryption oracle.

Recommendation

Stop claiming length decorrelation for v8, and either document the residual channel honestly or remove it. If the noise layer is kept as it is, the paper should state as a v8-specific caveat that the ciphertext length depends on message content and that, under chosen or merely varied associated data, an observer can recover the plaintext length bucket reliably, which is a strengthening of the already-documented CAV-003. If length leakage is to be avoided, make the token count N_{tok} independent of the nonce, for example by deriving the noise schedule only from the fixed padded length, or by padding N_{tok} up to a per-bucket ceiling so that $|C|$ depends only on the length bucket. The underlying tension should also be acknowledged: combining a message-derived, SIV-style nonce with a nonce-dependent, variable-length noise-expansion layer is inherently at odds with length hiding, and the lemma "Ciphertext length is decorrelated from plaintext content" must be dropped or restated for v8 regardless of which mitigation is chosen.

Client Comment

/napseq-eprint-v2.tex

```

382 \texttt{0x02} & Noise probability & (none) & $[0.75, 0.99]$ \\
959 \begin{definition}[Synthetic nonce --- domain
    \texttt{0x0A}]\label{def:synthnonce}
1121 $\mathsf{kb}\to\mathsf{sk}$, since neither depends on how the nonce
    is
1777 \label{lem:tar}

```

5.2. Moderate

CVF-3: open

Category: **algorithm**

Key-space and min-entropy figures are numerically wrong and internally contradictory

The Key subsection states the ordered 10-tuple key space as $|\mathcal{P}|!/(|\mathcal{P}| - 10)! \approx 1.1 \times 10^{58} \approx 2^{196}$ and $H_\infty(\mathbf{k}) \approx 196$ bits for $K = 10$. With the paper's own $|\mathcal{P}| \approx 586,000$ (so $\log_2 |\mathcal{P}| \approx 19.16$), the correct figures are $\approx 4.8 \times 10^{57} \approx 2^{191.6}$, i.e. $H_\infty \approx 19.16 \cdot 10 = 191.6$ bits. So 1.1×10^{58} is about $2.3\times$ too large and 2^{196} is wrong; even the paper's own (too-large) 1.1×10^{58} is only $2^{192.8}$.

This is internally contradictory: the remark "Minimum K for a target security level" uses the correct $H_\infty \approx 19.16 K$, which gives 191.6 bits at $K = 10$, flatly contradicting the "196 bits" stated in the Key subsection and repeated as " $K = 10$ ($H_\infty \approx 196$ bits)". Downstream, the post-quantum section's 2^{196} classical and 2^{98} post-Grover key-search figures should be $\approx 2^{191.6}$ and $\approx 2^{95.8}$. The ≈ 128 -bit post-Grover target and the $K \geq \lceil 128/19.16 \rceil = 7$ floor already use 19.16 and are unaffected, so the security margin holds; only the stated numbers are inconsistent.

Recommendation

Replace $1.1 \times 10^{58} \approx 2^{196}$ with $\approx 4.8 \times 10^{57} \approx 2^{191.6}$ and " ≈ 196 bits" with " ≈ 191.6 bits" in the Key subsection, reconcile it with the $19.16 K$ formula in the remark "Minimum K ...", and correct the post-quantum figures ($2^{196} \rightarrow 2^{191.6}$, $2^{98} \rightarrow 2^{95.8}$). Optionally reconcile the 2^{197} – 2^{257} range quoted elsewhere with the $19.16 K$ formula.

Client Comment**/napseq-eprint-v2.tex**

```

328  \frac{|\mathcal{P}|!}{(|\mathcal{P}|-10)!} \approx 1.1 \times
    10^{58} \approx 2^{196}.
339  ($\approx 196$ bits for $K=10$, $\approx 19$ bits for $K=1$). Because
1869 $\approx K \log_2 |\mathcal{P}| \approx 19.16 \log_2 |\mathcal{P}| \approx
    |\mathcal{P}| \approx
1878 $K=10$ ($H_\infty \approx 196$ bits) exceeds this minimum with margin.
2710 exponent, giving a post-quantum key-search complexity of $\approx
    2^{98}$,
2760 $\approx 2^{196}$. Grover's algorithm halves the exponent, giving a

```

CVF-4: openCategory: **algorithm**

Ciphertext-expansion range 4–20x is inconsistent with the noise-probability ceiling of 0.99

Noise is injected as a geometric run before each real token, so the expected token expansion is $1/(1-p)$ per real token. With the stated noise probability $p = 0.75 + 0.24t \in [0.75, 0.99]$, the expansion is $4\times$ at $p = 0.75$, $20\times$ at $p = 0.95$, and $100\times$ at the stated ceiling $p = 0.99$. The paper repeatedly claims 4–20 $\times$ (comparison table, primary trade-off text, and conclusion), but $20\times$ corresponds to $p = 0.95$, not the ceiling $p = 0.99$. Moreover the sentence stating the expansion is “mitigated ... by the noise-probability ceiling of 0.99” points at the worst case ($100\times$), not a mitigation.

Recommendation

Make the two figures consistent: either set the noise-probability ceiling to 0.95 (matching the 4–20 $\times$ claim) or state the expansion as 4–100 $\times$ (matching the 0.99 ceiling), and correct the “mitigated by the ceiling of 0.99” wording, since $p = 0.99$ is the maximal-expansion case.

Client Comment**/napseq-eprint-v2.tex**

```

382 \texttt{0x02} & Noise probability & (none) & $[0.75, 0.99]$ \\
2789 Ciphertext expansion & $4\text{--}20\times$ & $1\times$ & $1\times$ &
      $1\times$ \\
2822 mechanism produces $4\text{--}20\times$ ciphertext expansion
      depending on the
2827 of 0.99.
3014 produces $4\text{--}20\times$ ciphertext expansion relative to

```

5.3. Minor**CVF-5: open**Category: **procedural**

Domain-separation remark does not formally cover the synthetic-nonce domain 0x0A

The remark "Domain Separation of HMAC Inputs" is stated for domains $d \in \{0x00, \dots, 0x09\}$ with the single uniform input shape $d \parallel N \parallel \text{ctx}$ and the nonce always at byte offset 1..16. The v8 synthetic-nonce domain 0x0A is new and has a different input shape, $0x0A \parallel \text{be4}(|A|) \parallel A \parallel M$, with no nonce at bytes 1..16 (it produces the nonce). The cross-domain argument (distinct first byte) still covers it and intra-domain injectivity holds via the $\text{be4}(|A|)$ prefix, so there is no attack, but the remark and the hiding lemma's "every input shares the single uniform $d \parallel N \parallel \text{ctx}$ shape" sentence are formally stated for a domain set and input shape that v8's use of 0x0A under sk no longer matches. The extension is valid but should be stated.

Recommendation

Extend the remark "Domain Separation of HMAC Inputs" to domain 0x0A: note that its first byte differs from all other domains (unconditional cross-domain distinctness) and that $\text{be4}(|A|) \parallel A \parallel M$ is injective in (A, M) , and adjust the hiding lemma's "single uniform shape" wording to account for the 0x0A input shape.

Client Comment**/napseq-eprint-v2.tex**

```

1220 \begin{remark}[Domain Separation of HMAC Inputs]\label{rem:domsep}
1660 \]

```

CVF-6: openCategory: **readability**

Abstract and Contribution 1 state a non-injective synthetic-nonce formula (missing the AAD length prefix)

The abstract and Contribution 1 write the synthetic nonce as $N = \text{HMAC}(\text{sk}, 0x0A \parallel A \parallel M)$, without the $\text{be4}(|A|)$ length prefix that the normative "Synthetic nonce" definition includes. As written, $0x0A \parallel A \parallel M$ is not injective in (A, M) : for example $(A, M) = (x, y)$ and (ε, xy) produce the same input and hence the same nonce, so an adversary who controls the AAD split could force a deterministic nonce collision across distinct (A, M) pairs. The normative definition includes $\text{be4}(|A|)$ and is fine; only the abbreviated formula is wrong and should match.

Recommendation

Correct the abstract and Contribution 1 to $N = \text{HMAC}(\text{sk}, 0x0A \parallel \text{be4}(|A|) \parallel A \parallel M)$, matching the normative "Synthetic nonce" definition, so the informal formula is injective in (A, M) .

Client Comment**/napseq-eprint-v2.tex**

```

66 synthetically as $N=\mathrm{HMAC}(\mathrm{sk}, \texttt{0x0A}\|A\|M)$
    in the
140 message-bound nonce
    $N=\mathrm{HMAC}(\mathrm{sk}, \texttt{0x0A}\|A\|M)$.

```

CVF-7: openCategory: **naming**

The symbol N is overloaded: 16-byte nonce versus real-plus-noise token count

In the Wire Format (Version 7) subsection, N denotes the token count in " $|B| = 8N$ where N (real + noise tokens)" and, two lines later, the nonce in " $C = N \parallel \widetilde{B} \parallel T$, where N is a 16-byte random nonce". The same clash appears in the AEAD-triple ciphertext-space definition ("length $16 + 8N + 32 \dots N$ is the real-plus-noise token count"). The traffic-analysis lemma avoids this by writing N_{tok} , but the normative wire-format text does not, creating a reproducibility ambiguity.

Recommendation

Rename the token count to N_{tok} (or similar) throughout the wire-format and ciphertext-space text, reserving N for the nonce.

Client Comment**/napseq-eprint-v2.tex**

```

601 so  $\lvert B \rvert = 8N$  where  $N$  (real + noise tokens) depends
    only on
608 where  $N$  is a 16-byte random nonce and  $\widetilde{B}$  is the
804     real-plus-noise token count for that message (minimum 48
        bytes).
```

CVF-8: openCategory: **behavior**

LEB128 canonicity rule does not fully enforce $x < 2^{64}$

The "Unsigned LEB128 encoding" definition requires a decoder to reject "any input requiring more than $\lceil 64/7 \rceil = 10$ groups" and claims the resulting map is a bijection on $[0, 2^{64})$. But a 10-group LEB128 value carries up to 70 bits, so canonical 10-group encodings represent integers up to $2^{70} - 1$, including values in $[2^{64}, 2^{70})$ that the definition simultaneously declares out of range. The group-count check alone therefore admits some $x \geq 2^{64}$, and the claimed bijection on $[0, 2^{64})$ does not hold. Impact is low for block-mode tokens (fixed-width be8), but the streaming format still uses varint, so the decoder rule matters there.

Recommendation

Add a top-group bound to the decoder rule: in the 10th group only the least-significant bit (bit 63) may be set, i.e. reject any 10-group encoding whose value is $\geq 2^{64}$. This restores the bijection on $[0, 2^{64})$.

Client Comment**/napseq-eprint-v2.tex**

```

307     $\lceil 64/7 \rceil = 10$ groups, and truncated input (a
311 bijection on  $[0, 2^{64})$ , eliminating the ambiguity (multiple valid

```

CVF-9: openCategory: **documentation**

The HMAC-SHA256 PRF advantage is never formally defined, and its use on truncated outputs is not justified

The security theorems bound advantages in terms of $\text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}$ but never state a formal PRF definition (oracle, distinguisher, advantage). In addition, several load-bearing values are truncated HMAC outputs — the synthetic nonce uses $[0:16]$ (128 bits) and the derivation primitives use $[0:8]$ and $[0:4]$ — yet the paper invokes the full-output HMAC-SHA256 PRF advantage without noting that truncating a PRF preserves the PRF property (with the same or negligibly different advantage). The bounds are not violated, but the assumption as actually used (a truncated-output PRF) is never made precise. This continues the prior-round observation that the PRF assumption is used throughout but never formally stated.

Recommendation

State the PRF definition and advantage Adv^{PRF} explicitly, and add a sentence noting that for any fixed output truncation τ , a distinguisher against the truncated PRF yields one against full HMAC-SHA256 with the same advantage, so the truncated derivations (nonce $[0:16]$, addend/char $[0:4]$, threshold $[0:8]$) are covered by the stated assumption.

Client Comment**/napseq-eprint-v2.tex**

```

480 \label{def:isnoise}

```



```

964      \texttt{0x0A} \, \, \, \, \mathrm{be4}(|A|) \, \, \, \, A \, \, \, \,
      M\biggr)[0{:}16].
998 \label{lem:v8-cvf3}
1404 \begin{theorem}[IND-CPA]\label{thm:ind-cpa}

```

CVF-10: openCategory: **documentation**

"Minimum valid ciphertext length is 48 bytes" understates the true minimum

The decryptor rejects $|C| < 48$ (16-byte nonce plus 32-byte tag) and the text calls 48 the "minimum valid ciphertext length". But every message, including the empty one, pads to $B = 16$ codepoints plus the 2-codepoint length prefix, i.e. at least 18 real tokens, so $|\widetilde{B}| \geq 8 \cdot 18 = 144$ bytes and the minimum length of a well-formed ciphertext is $\approx 16 + 144 + 32 = 192$ bytes (larger once noise tokens are added). 48 is a safe parse floor, not the minimum of a valid ciphertext; the wording overstates what 48 guarantees.

Recommendation

Clarify that 48 bytes is the parse-time rejection floor (nonce plus tag), and that the minimum length of a well-formed ciphertext is ≈ 192 bytes ($16 + 8 \cdot 18 + 32$), before noise expansion.

Client Comment**/napseq-eprint-v2.tex**

```

654 reference code actually computed). The minimum valid ciphertext
      length is
804      real-plus-noise token count for that message (minimum 48
      bytes).

```

CVF-11: openCategory: **documentation**

Streaming format retains the CVF1 codepoint-length leak (by design) but does not document it

The online-AE streaming format masks a varint blob (not the fixed-width be8 encoding used by block mode), so the token byte-length still grows with codepoint value — the exact channel that CVF1 closed for block mode. This is consistent with the stated rule that a streaming ciphertext always discloses the exact plaintext length, so it is not a new confidentiality break, but the paper never explicitly says the CVF1 length leak is knowingly retained (and mooted) in streaming mode. A reader could wrongly assume the CVF1 fix is universal.

Recommendation

Add a one-line caveat to the streaming-format section stating that it deliberately uses varint and therefore retains the CVF1 codepoint-length leak, which is acceptable because a streaming ciphertext already discloses the exact plaintext length.

Client Comment

/napseq-eprint-v2.tex

```
663 eliminates this by attaching a per-chunk HMAC-SHA256 tag to each
    segment of the
668 \]
```

CVF-12: open

Category: **procedural**

Legacy v7 is retained with security theorems the paper itself flags as conditional or unproven

The paper keeps the single-secret v7 construction "for backward compatibility only" and states its IND-CPA, INT-CTXT, and IND-CCA theorems, but its own remarks concede that the v7 reductions are not simulatable as written (the CVF13 simulation gap: a reduction that must run the arithmetic layer needs $\mathbf{k}$, which is also the HMAC key it is meant to distinguish from random) and that the v7 PRF term rests on a non-standard, adversary-conditioned key distribution (the CVF8 residual and the D -conditioned PRF remark). The IND-CPA theorem is therefore explicitly "read as conditioned on this gap". Retaining a construction whose headline theorems are admittedly conditional, alongside v8 as the recommended default, is a documentation and soundness hazard: a reader may treat the v7 theorems as unconditional.

Recommendation

Either remove the legacy v7 construction and its theorems from the paper, or clearly quarantine them: mark each v7 theorem in its statement (not only in a remark) as

conditional on the unresolved CVF13 simulation gap and the CVF8 non-standard key-distribution assumption, and state prominently that only v8 carries unconditional guarantees.

Client Comment

/napseq-eprint-v2.tex

```

1169 \label{rem:v8-scope}
1404 \begin{theorem}[IND-CPA]\label{thm:ind-cpa}
1505 \label{rem:prf-d}
1528 \label{rem:cvf13}

```

CVF-13: open

Category: **architecture**

The paper should drop the legacy v7 construction entirely rather than carry two parallel schemes, now that v8 is the recommended default

The document specifies two complete, non-interoperable constructions side by side: the legacy single-secret v7 scheme, including its algorithm triple, its wire format, and its full IND-CPA, INT-CTXT, and IND-CCA proofs, and the v8 scheme, which the abstract and the scope remark now designate as the primary and recommended construction. Keeping both in a single ePrint roughly doubles the specification surface and the proof burden, and it forces every reader to keep track of which theorem, remark, and caveat belongs to which of the two schemes.

This is more than an aesthetic cost. The v7 scheme is retained together with a security proof that the paper itself flags as conditional, namely the CVF13 simulation-gap caveat and the non-standard key-entropy assumption, and together with a documented catastrophic failure mode in which a single nonce reuse yields exact key recovery. Presenting a scheme with those properties next to the recommended one, under the same title and in the same normative text, invites a reader either to deploy v7 or to carry v7's theorems over to v8. Because v8 does not reuse v7's key schedule, and because the paper already states that v7 is retained for backward compatibility only, v7 contributes nothing that v8 needs.

This finding is the structural counterpart to the separate observation that v7's theorems are presented as though they were unconditional. That other finding argues the soundness hazard of leaving the conditional theorems in place; this one argues that the cleanest remedy is to remove the v7 construction from the paper altogether.

Recommendation

Remove the legacy v7 construction and its proofs from the ePrint, and let v8 be the single scheme the paper specifies and analyses. If a record of v7 is still needed for existing deployments, relocate it to a clearly separate legacy document or a short historical appendix that is explicitly marked as deprecated and outside the scope of the security claims, rather than interleaving it with the recommended construction. Doing so also shrinks the proof surface that a reader must check and removes the temptation to treat v7's conditional theorems as current guarantees.

Client Comment

/napseq-eprint-v2.tex

```

767 \label{sec:algorithm-triple}
905 \subsection{The NAPQES Construction, Version 2 (Default): Independent
    Key Schedule and Synthetic Nonce (CVF3 / CVF8 / CVF13 fix)}
1169 \label{rem:v8-scope}
1404 \begin{theorem}[IND-CPA]\label{thm:ind-cpa}

```

CVF-14: open

Category: **readability**

The audit-response and revision-history remarks should be moved out of the normative text into a separate companion document

The specification is interleaved with a large amount of revision-history and audit-response material. Throughout the construction and the security analysis there are on the order of twenty-five dedicated remark blocks titled "Audit finding CVFn", "Erratum", "Retracted", or "Retitled", together with roughly ninety inline references of the form "this was audit finding CVFn" or "(CVFn fix)", as well as section and lemma titles annotated with fix numbers. This material documents how the current version differs from earlier drafts; it is not part of the construction, and no proof relies on it.

Its presence carries two costs. First, it makes the normative text substantially harder to read, because a reader who only wants the current design must continually separate the specification itself from the account of what changed between drafts. Second, because the text repeatedly asserts that each prior problem has been fixed, it nudges a reader, and even a reviewer, toward accepting those assertions without independently checking the current text, which is exactly the confirmation bias a careful auditor tries to avoid. In practice a clean reading copy of the paper, with all of this narration removed, is materially easier to evaluate on its own merits.

Recommendation

Move the audit-response and revision-history remarks into a separate companion document, for example a CHANGES or audit-response file that maps each prior finding to the corresponding place in the current text. In the main paper, keep only the load-bearing definitions, lemmas, and theorems, giving them neutral names rather than fix-numbered labels, and retain in the body only those few remarks that state a caveat still in force for the current construction, reworded as plain caveats without the finding reference. The result is a specification that reads as a self-contained description of the current scheme rather than as a changelog.

Client Comment

/napseq-eprint-v2.tex

```

256 \begin{remark}[Audit finding CVF17: LEB128 used without a definition
    or normative reference]
350 \begin{remark}[CVF2 fix: unified domain-first layout]
554 \begin{remark}[Retracted: the GCD claim below was vacuous (audit
    finding CVF4)]
1527 \begin{remark}[Audit finding CVF13:  $\mathcal{B}_1/\mathcal{B}_2$ 
    cannot simulate NAPQES encryption from oracle access alone]
1715 \begin{remark}[Erratum: the v6 argument was false]

```

6. Conclusion

The v2 review of the NAPQES ePrint preprint identified fourteen issues: two of high severity, two of moderate severity, and ten of low severity. Of the thirty-two findings raised in the first round, twenty-seven are fixed, three are partially fixed, and two remain unresolved by design; the fourteen findings below are the surviving residuals, the newly introduced residuals, and the new observations arising from the v2 restructuring. As specified, the revised scheme materially improves on the first round but still does not fully substantiate the confidentiality goal it claims for its recommended default mode.

The two high-severity findings concern the security of the version-8 default. The IND-CPA theorem is stated for a deterministic scheme against the standard left-or-right game in which the adversary may query the challenge message; because encryption of a fixed (A, M) pair is a pure function, this permits a trivial distinguishing attack, so the theorem is false as stated and should instead be proved against a misuse-resistant deterministic-AEAD notion. Separately, the version-8 ciphertext length still depends on plaintext content through the power-of-two padding bucket, and because the synthetic nonce is deterministic an adversary who varies the associated data can collect repeated encryptions of the same message, averaging away the interleaved noise and turning the acknowledged length-bucket leak into a reliable bucket oracle that contradicts the hiding claim of `lem:tar`.

The two moderate-severity findings are quantitative errors in the revised analysis. The key-space and min-entropy figures are numerically wrong and internally inconsistent, and the post-quantum Grover figures (2^{98} and 2^{196}) do not match the stated key entropy; and the advertised $4\text{--}20\times$ ciphertext-expansion range is inconsistent with the geometric noise process at the maximum noise probability 0.99, which admits substantially larger expansion.

The ten low-severity findings collect specification and proof-hygiene gaps: the domain-separation remark does not cover the new `0x0A` synthetic-nonce input; the abstract advertises a non-injective synthetic-nonce formula; the symbol N is overloaded between the 16-byte nonce and the real-plus-noise token count; the new LEB128 canonicity rule bounds the group count but not the top group, so values $\geq 2^{64}$ are still accepted; the HMAC-SHA256 PRF advantage is used throughout but never formally defined and its truncated-output use is unaddressed; the “minimum valid ciphertext length” wording understates the true minimum; the streaming format silently retains the first round’s codepoint-length leak; and the legacy version-7 construction is retained together with security theorems the paper itself flags as unproven. The review additionally recommends dropping the legacy version-7 construction entirely and moving the audit-response and revision-history remarks out of the normative text into a companion document.

Client Comment



About Us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and coauthored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, Javascript, and other languages.

Contacts

Email

info@abdkconsulting.com

Website

abdk.consulting

X (former Twitter)

[@ABDKConsulting](https://twitter.com/ABDKConsulting)

LinkedIn

company/abdk-consulting